

Led Tower

Francisco Meireles
Faculdade de Engenharia
Universidade do Porto
Porto, Portugal
up202007382@up.pt

João Legoinha
Faculdade de Engenharia
Universidade do Porto
Porto, Portugal
up202004912@up.pt

Tiago Catalão
Faculdade de Engenharia
Universidade do Porto
Porto, Portugal
up201404374@up.pt

Abstract—This report describes the development process of the LED Tower Project, a real-time system that implements Snake 3D and various visual animations on a 6x6x6 3D LED grid. The project utilizes an ESP32 microcontroller running FreeRTOS to manage multiple tasks, including input handling, output control, and animation logic. The system employs MCP23017 GPIO expanders for I/O management and 74HC164 shift registers for controlling the LED grid.

I. GENERAL EXPLANATION

INTRODUCTION

This report describes the process behind the development of the LED Tower Project, undertaken as part of the Embedded Systems course. The main objective of the project was to apply the theoretical concepts of Real-Time Operating Systems (RTOS), discussed during the course, to a real-world application. By designing and implementing a system capable of managing concurrent tasks with precise timing, the project aimed to demonstrate the practical use of RTOS principles in embedded systems.

HARDWARE SPECIFICATIONS

The hardware setup for the project consisted of an ESP32 microcontroller running FreeRTOS, a widely used real-time operating system. The ESP32 was chosen over alternative ESP8266 due to its ability to support both FreeRTOS and the Arduino platform and libraries simultaneously, providing greater flexibility and simplicity, and computational power for real-time applications.

FreeRTOS simplifies the development of the real-time applications by providing advanced task management features, such as scheduling and prioritization. Combined with the ESP32's computational power and a clock frequency ranging from 80MHz to 240MHz, this duo, provided an ideal environment for developing parallel tasks with precise timing requirements.

The system's output was a 6x6x6 LED cube, where each LED could be individually controlled, enabling the creation of complex visual patterns and real-time interactivity. To manage the LED grid, the project utilized three MCP23017 GPIO expanders combined with two 74HC164 shift registers for controlling the LEDs. Additionally, the MCP23017 GPIO expanders were responsible for input handling.

The ESP32 communicates with both MCP23017 GPIO expanders and 74HC164 shift registers using the I2C communication protocol in fast mode (400Kbps).

OBJECTIVES

The primary goal of the project was to create an interactive real-time application, capable of managing concurrent tasks, including frame rendering for animations, user input handling, and game logic execution, while applying the following basic RTOS principles:

- Task Scheduling: Ensure that different tasks are executed at precise intervals and within their timing restraints.
- Task Prioritization: Assigning higher priorities to critical tasks, such as user interactions, to maintain system responsiveness.
- Timing and Synchronization: Utilizing FreeRTOS's mechanisms to maintain smooth transitions and consistent animations.

To demonstrate the system's capabilities, the project implemented several animations and a game:

Animations

- Rain Effect: Simulates falling raindrops on the LED grid.
- Arrows: Displays arrows that originate from each vertex of the cube and converge towards the center.
- Resizing Cube: Animates a cube that grows and shrinks in size.

Game

- Snake 3D: A 3D adaptation of the classic Snake game, where the player controls a snake that grows as it consumes targets within the LED grid.

II. IMPLEMENTATION AND SOFTWARE ARCHITECTURE

Task Description

The system was divided into three main tasks, each responsible for a specific functionality:

- Input Task:
 - This task is responsible for reading the input states from the MCP23017 GPIO expanders. It continuously polls the GPIO states and updates the buttons array with the current state of the buttons and their changes (falling and trailing edge detection)
- Output Task:
 - This task is responsible for updating the output states of the MCP23017 GPIO expanders based on the current state of the grid. It writes the grid data to

the GPIO expanders to control the LEDs for a single LED horizontal plane at each execution.

- Animations Task:
 - This task is responsible for handling animations and game logic. It updates the grid state based on the current animation or game rules and inputs and communicates with the output task to render the changes. The task also supports dynamic switching between animations through button inputs, allowing users to interact with the system in real-time.

The period (and deadline) of each task is summarized in Table I.

TABLE I
TASK PERIODS AND DEADLINES

Task	Period = Deadline (ms)
Output Task	1
Input Task	50
Animations: Rain	200
Animations: Cube	200
Animations: Arrow	200
Animations: Snake	400

The period for the tasks was empirically chosen. With a period of $2ms$ for the output task a noticeable flicker in the LEDs was present, which disappeared with a smaller period. The animations task period only affected the animations speed, and a value of $200ms$ proved to be an acceptable speed. For the input task, its period requirements is to only be small enough to register a button press or release without a noticeable delay for the user, which is fulfilled with a period of $50ms$.

Tasks Execution Times

The execution times for each task were measured individually over 500 iterations using the `micros()` function to avoid interference from other tasks. This ensures that the measurements reflect the actual time required for each task to complete its work in isolation. The following table summarizes the periods and execution times for each task:

TABLE II
TASKS EXECUTION TIMES

Task	Execution Times (μs)		
	Min	Avg	Max
Output	80	453	724
Input	1666	1675	1768
Animations: Rain	87	87	121
Animations: Cube	71	71	138
Animations: Arrow	1	1	181
Animations: Snake	2	2	61

Rate Monotonic Scheduling (RMS)

To ensure that all tasks meet their timing requirements, the system uses Rate Monotonic Scheduling (RMS), a fixed-priority scheduling algorithm. In RMS, tasks with shorter periods are assigned higher priorities:

- The priority order for the tasks is as follows:
 - Output Task: Highest priority (shortest period: 1 ms).
 - Input Task: Medium priority (period: 50 ms)
 - Calculation Task: Lowest priority (longest period: 200 ms).

CPU Utilization Analysis

To evaluate the schedulability of the system under Rate Monotonic Scheduling, the utilization-based test Least Upper Bound was used. This test provides a sufficient condition to guarantee that all tasks meet their deadlines, ensuring that at least one instance of each task is executed within its respective period. The test is based on the following condition:

$$U(n) = \sum_{i=1}^n \frac{C_i}{T_i} \leq n \left(2^{\frac{1}{n}} - 1 \right) \quad (1)$$

Where:

- C_i : Execution time of task i .
- T_i : Period of task i .
- n : Number of tasks in the system.

For the Animations Task, we consider the worst-case scenario across all animations, Arrow animation:

$$C_1 = 0.724ms, T_1 = 1ms, \frac{C_1}{T_1} = 0.724 \quad (2)$$

$$C_2 = 1.768ms, T_2 = 50ms, \frac{C_2}{T_2} = 0.03536 \quad (3)$$

$$C_3 = 0.181ms, T_3 = 200ms, \frac{C_3}{T_3} = 0.00069 \quad (4)$$

Summing up the utilizations:

$$U(n) = \sum_{i=1}^n \frac{C_i}{T_i} = 0.724 + 0.03536 + 0.00069 = 0.7603 \quad (5)$$

For $n = 3$ tasks, the theoretical utilization bound is given by:

$$n \left(2^{\frac{1}{n}} - 1 \right) = 3 \left(2^{\frac{1}{3}} - 1 \right) = 0.7798 \quad (6)$$

Since the calculated utilization $U(n) = 0.7603$ is less than the theoretical upper bound 0.7798 , it can be concluded that the system is schedulable under RMS. This means that all tasks will meet their deadlines, ensuring the proper operation of the real-time system.

Response Time Analysis

To further validate the schedulability of each task, we use an iterative technique to calculate the Response Time Worst-Case (RWCi) for each task. The iterative process continues until the response time converges (i.e., $Rwc_i(m+1) = Rwc_i(m)$) or diverges (i.e., $Rwc_i(m+1) > D_i$, where D_i is the task's deadline that in this case is equal to the period).

The response time is calculated using the following iterative equations:

$$Rwc_i(0) = \sum_{k \in hp(i)} C_k + C_i$$

$$Rwc_i(m+1) = \sum_{k \in hp(i)} \left\lceil \frac{Rwc_i(m)}{T_k} \right\rceil \cdot C_k + C_i$$

where:

- C_k : Execution time of a higher-priority task k .
- C_i : Execution time of the current task i .
- T_k : Period of task k .
- $hp(i)$: Set of tasks with higher priority than task i .

The worst-case response times for each task, calculated using the Max values from Table 1, are summarized below.

TABLE III
WORST-CASE RESPONSE TIMES

Task	Worst-Case Response Time (μs)
Output Task	724
Input Task	3487
Animations Task	7017

The analysis shows that all tasks meet their deadlines:

- The Output Task has a worst-case response time of 724 μs , well within its deadline of 1 ms.
- The Input Task has a worst-case response time of 3487 μs , significantly below its deadline of 50 ms.
- The Animations Task has a worst-case response time of 7017 μs , which is well within its deadline of 200 ms.

Since the worst-case response times for all tasks are less than their deadlines, the system is schedulable under Rate Monotonic Scheduling.

III. RESULTS

To evaluate the system's performance under real-world conditions, the execution times of all tasks were measured while running concurrently. This provides insight into how task scheduling and prioritization impact the system's overall behavior.

TABLE IV
TASKS EXECUTION TIMES

Task	Execution Times (μs)		
	Min	Avg	Max
Output	93	472	721
Input	1710	1814	1932
Animations: Arrow	4	11	213

The results show that the execution times measured under concurrent operation are slightly higher than the measured values from isolated execution, as is expected. This is due to the overhead introduced by task scheduling, context switching, and potential interference between tasks.

Despite these variations, the measured execution times remain well within the tasks' respective deadlines and are still lower than the worst-case response times calculated in Table III. This confirms that the system operates as expected under concurrent conditions, even in the most demanding scenarios.

IV. CONCLUSIONS

This report described the design and implementation of a real-time system for controlling and animating a 3D LED grid using an ESP32 microcontroller and FreeRTOS. The system successfully demonstrated the application of RTOS concepts, such as task scheduling, prioritization, and synchronization, to manage concurrent tasks effectively. The implementation of a robust architecture ensured reliable control of the LED grid, while timing validation through Rate Monotonic Scheduling and worst-case response time analysis confirmed that all tasks meet their deadlines.

The system's ability to integrate interactive features, such as animations and a 3D Snake game, showcased its versatility and responsiveness. These functionalities not only met the project's technical goals but also highlighted its potential for future extensions, such as adding more animations or games. A demonstration of the project is available in [1].

Overall, the project provided a practical example of embedded systems and RTOS principles in action, achieving its objectives and demonstrating the feasibility of real-time control in complex systems.

V. DEDICATION OF EACH MEMBER

Francisco Meireles: 33.3%

João Legoinha: 33.3%

Tiago Catalão: 33.3%

REFERENCES

- [1] Tiago Catalão Francisco Meireles, João Legoinha. Feup semb (2024/2025) - led tower. <https://www.youtube.com/watch?v=YuEzPVUi9ik>.
- [2] Luís Almeida. Embedded systems - real-time scheduling, part 4 fixed priorities scheduling, 2015.